

SMC Boot ROM — Low-Level Design

SEP Boot ROM — Low-Level Design

Document: 02_BOOTROM_LowLevel_Design.md **Module:** smc::bootrom **Companion docs:** -
01_BOOTROM_Specification.md — external contract - 03_BOOTROM_Test_Plan.md — verification plan

Contents

1. Scope
 2. Source-tree layout
 3. Module structure
 - 3.1 Ports
 - 3.2 Public methods (API)
 - 3.3 Internal methods and SC processes
 - 3.4 File-local helpers
 - 3.5 Function call flow
 4. CCI parameter catalogue
 5. Constructor walkthrough
 6. Internal data structures
 7. TLM callback implementation
 8. Preload parser
 9. Debug back-door API
 10. Reset handling
 11. Error / fatal-report taxonomy
 12. Single-driver discipline
 13. Performance & memory footprint
 14. Build & packaging
 15. Coding-style notes
 16. Future work
-

1. Scope

This document covers the **internal** SystemC implementation of the SEP Boot ROM model — the data layout, the TLM callbacks, the preload parser, and the CCI scaffolding. The externally-visible contract (register map, error responses, preload formats) is documented in 01_BOOTROM_Specification.md and is not repeated here.

The Boot ROM is the simplest IP in the SMC peripheral suite, so this document is correspondingly short — but the conventions it follows (CCI declaration order, single-driver rule, reset-as-no-op, error-report taxonomy) mirror exactly what the PLIC and CLINT do, so a reader familiar with either of those documents will recognise the pattern.

2. Source-tree layout

```

bootrom/
├── CMakeLists.txt           Top-level project; libsmc_bootrom + tests
├── README.md
├── run_tests.sh             Build + run helper (Release / ASan / coverage)
├── include/
│   ├── smc_tlm_extensions.h Shared GP extension (smc_axi_extension)
│   └── bootrom.h            SC_MODULE(bootrom) + cci_param declarations
├── src/
│   └── bootrom.cpp           Implementation
├── test/
│   ├── CMakeLists.txt
│   ├── bootrom_tb.cpp        Primary test bench (hex preload)
│   ├── bootrom_bin_tb.cpp    Secondary test bench (binary preload)
│   └── fixtures/
│       ├── bootrom_sanity.rv64.hex Hex preload fixture
│       └── bootrom.rv64.img      Binary preload fixture (verbatim from
│                               tt-oca-hw/hw/smc/data/scripts/)
└── doc/
    ├── 01_BOOTROM_Specification.md External contract (RDL-traceable)
    ├── 02_BOOTROM_LowLevel_Design.md (this file)
    ├── 03_BOOTROM_Test_Plan.md      Verification plan
    ├── build_docs.sh                Markdown → PDF script
    └── print.css                     PDF stylesheet (shared with CLINT)

```

When the rest of the SMC IP library exists, drop `include/` and `src/` into `libsmc/bootrom/` and replace the local `smc_tlm_extensions.h` with the project-wide canonical version (the file is bit-identical across the bootrom, plic, and clint sub-packages).

3. Module structure

```
#include <cci_configuration>

namespace smc {

class bootrom : public sc_core::sc_module {
protected:
    // CCI params declared FIRST so they are constructed before any
    // member that might read them (here: data_ is sized from
    // size_bytes_p_.get_value() during construction).
    cci::cci_param<uint64_t, cci::CCI_IMMUTABLE_PARAM> size_bytes_p_;
    cci::cci_param<std::string, cci::CCI_IMMUTABLE_PARAM> init_file_p_;
    cci::cci_param<std::string, cci::CCI_IMMUTABLE_PARAM> init_file_format_p_;
    cci::cci_param<double> access_delay_ns_p_;

public:
    SC_HAS_PROCESS(bootrom);
    tlm_utils::simple_target_socket<bootrom> reg_socket;
    sc_core::sc_in<bool> rst_n_i;

    explicit bootrom(sc_core::sc_module_name name,
                    bootrom_cfg cfg = bootrom_cfg{});

    // Debug API ...
};

} // namespace smc
```

The four CCI params are `protected` because they are model-internal configuration, not part of the bus contract. They sit ahead of every public port so that the C++ rule “members are initialised in declaration order” guarantees the CCI broker has already resolved `size_bytes` by the time `data_` is sized.

3.1 Ports

Port	Type	Dir	Purpose
<code>reg_socket</code>	<code>tlm_utils::simple_target_socket<bootrom></code>	target	TLM-2.0 read access (AXI / AXI-Lite-style initiator; typically the SEP core fetch port).
<code>rst_n_i</code>	<code>sc_core::sc_in<bool></code>	in	Active-low reset. No-op for a ROM (no mutable state); kept for IP-suite symmetry.

3.2 Public methods (API)

Method	Signature	Purpose
constructor	<code>bootrom(sc_module_name, bootrom_cfg = {})</code>	Build, validate config, size + optionally preload <code>data_</code> , register callbacks/process (see §5).
<code>dbg_read64</code>	<code>uint64_t dbg_read64(uint64_t off) const</code>	Back-door aligned 64-bit read; returns 0 if OOB/unaligned. No bus delay.
<code>dbg_read32</code>	<code>uint32_t dbg_read32(uint64_t off) const</code>	Back-door aligned 32-bit read; returns 0 if OOB/unaligned.
<code>size_bytes</code>	<code>uint64_t size_bytes() const</code>	CCI-resolved total ROM size.
<code>dbg_load_bytes</code>	<code>unsigned dbg_load_bytes(uint64_t off, const uint8_t* ptr, unsigned len)</code>	Test-bench helper: copy <code>len</code> bytes into <code>data_</code> ; returns bytes written (0 if OOB).
<code>dump_state</code>	<code>void dump_state(std::ostream& = std::cout) const</code>	Human-readable config + first 32 bytes of contents.

3.3 Internal methods and SC processes

Member	Signature	Kind	Purpose
<code>b_transport</code>	<code>void b_transport(tlm::tlm_generic_payload&, sc_core::sc_time&)</code>	TLM <code>b_transport</code> callback	Blocking read path; writes are silently ignored (§7).
<code>transport_dbg</code>	<code>unsigned int transport_dbg(tlm::tlm_generic_payload&)</code>	TLM <code>transport_dbg</code> callback	Zero-delay back-door read; same write-ignore contract (§7, §9).
<code>reset_proc</code>	<code>void reset_proc()</code>	SC_METHOD (sensitive to <code>rst_n_i</code>)	Logs reset events; no state to clear (§10).
<code>resolve_format</code>	<code>static std::string resolve_format(const std::string& format, const std::string& path)</code>	static helper	Resolve "auto" to "hex" / "bin" from the filename suffix (§8).
<code>load_preload</code>	<code>void load_preload(std::vector<uint8_t>& const)</code>	private helper	Parse the preload image into <code>data_</code> ; <code>SC_REPORT_FATAL</code> on error (§8).
<code>parse_hex_line</code>	<code>static bool parse_hex_line(const std::string& line, uint64_t& value, bool& consumed)</code>	static helper	Parse one hex preload line; <code>consumed=false</code> for blank/comment lines (§8).

Processes/threads: the model registers exactly **one** SC process — `reset_proc` (`SC_METHOD`), which only logs. There are **no** `SC_THREAD` / `SC_CTHREAD` processes and no output signals to drive: all bus activity is driven synchronously through the `b_transport` / `transport_dbg` socket callbacks.

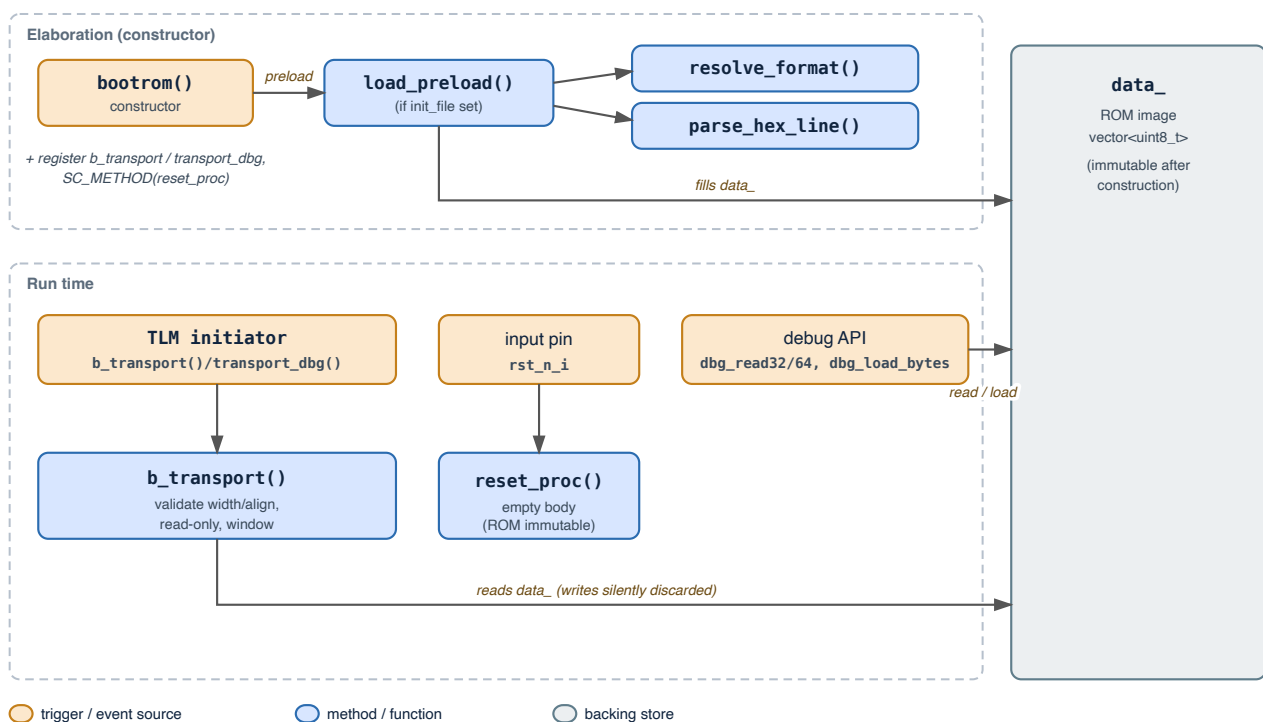
3.4 File-local helpers (bootrom.cpp anonymous namespace)

Function	Signature	Purpose
<code>to_lower</code>	<code>std::string to_lower(std::string s)</code>	Lowercase copy for case-insensitive matching.
<code>ends_with_ci</code>	<code>bool ends_with_ci(const std::string& s, const std::string& suffix)</code>	Case-insensitive suffix test; used by <code>resolve_format</code> to spot <code>.img / .bin</code> .
<code>is_supported_width</code>	<code>constexpr bool is_supported_width(unsigned len)</code>	True for <code>len ∈ {1,2,4,8}</code> .
<code>is_aligned</code>	<code>constexpr bool is_aligned(uint64_t addr, unsigned len)</code>	True iff <code>addr</code> is naturally aligned to <code>len</code> .

3.5 Function call flow

The Boot ROM has **no output ports and no recompute spine** — it is a passive TLM target that serves reads from a preloaded `data_` image. The diagram below shows the two phases: *elaboration* (the constructor fills `data_`) and *run time* (the socket callbacks serve reads).

Boot ROM — SystemC function-call flow



The Boot ROM is read-only: `b_transport / transport_dbg` serve reads from the preloaded `data_` image; writes return `TLM_OK` but are discarded. `reset_proc()` has an empty body (binding hook / extension point). `b_transport` annotates `access_delay_`; `transport_dbg` does not.

Boot ROM SystemC function-call flow: the constructor preloads the `data_` ROM image; at run time `b_transport / transport_dbg` serve reads (writes are discarded) and `reset_proc` is an empty binding hook.

Reading the flow:

- Elaboration** — `bootrom()` → (if `init_file` is set) `load_preload()` → `resolve_format()` + `parse_hex_line()` → fills `data_`. The constructor also registers the TLM callbacks and `SC_METHOD(reset_proc)`.

2. **Read** — TLM initiator → `b_transport()` validates width/alignment/streaming/byte-enables/window, then copies `data_[addr..]` and annotates `delay += access_delay_`. Writes return `TLM_OK` but are discarded (the ROM is read-only).
3. **Back-door** — `transport_dbg()` is a zero-delay read of `data_`; the debug API (`dbg_read32/dbg_read64/dbg_load_bytes`) accesses `data_` directly.
4. **Reset** — `reset_proc()` (`SC_METHOD` on `rst_n_i`) has an empty body; `data_` is immutable so there is nothing to clear (it is a binding hook / extension point).

4. CCI parameter catalogue

Param	C++ type	Mutability	Default ⇒ origin
<code>size_bytes</code>	<code>cci_param<uint64_t, CCI_IMMUTABLE_PARAM></code>	immutable	<code>cfg.size_bytes = 0x10000</code>
<code>init_file</code>	<code>cci_param<std::string, CCI_IMMUTABLE_PARAM></code>	immutable	<code>cfg.init_file = ""</code>
<code>init_file_format</code>	<code>cci_param<std::string, CCI_IMMUTABLE_PARAM></code>	immutable	<code>cfg.init_file_format = "auto"</code>
<code>access_delay_ns</code>	<code>cci_param<double></code>	mutable	<code>cfg.access_delay_ns = 1.0</code>

Each parameter has provenance metadata attached after construction:

```
size_bytes_p_.add_metadata("rdl_dimension",
    cci::cci_value(std::string("mem_array[NUM_ENTRIES]")));
size_bytes_p_.add_metadata("default_KiB", cci::cci_value(unsigned(64)));
init_file_p_.add_metadata("plusarg_equivalent",
    cci::cci_value(std::string("+sep_boot_rom_preload=<file>")));
init_file_format_p_.add_metadata("valid_values",
    cci::cci_value(std::string("hex|bin|auto")));
access_delay_ns_p_.add_metadata("unit", cci::cci_value(std::string("nanoseconds")));
access_delay_ns_p_.add_metadata("tlm_phase", cci::cci_value(std::string("annotated_delay")));
```

`rdl_dimension`, `default_KiB`, and `plusarg_equivalent` are the same metadata keys used by the PLIC and CLINT models so a single introspection tool can walk every SMC IP uniformly.

Why these mutabilities:

- **size_bytes** — changing it after elaboration would force a `data_.resize()`, invalidating any pointers the upstream initiator has cached. Made immutable.
- **init_file**, **init_file_format** — referenced only by the constructor. Marking them mutable would be meaningless (the parser has already run by the time mutation could happen).
- **access_delay_ns** — re-read on every `b_transport`, so a CCI mutation takes effect on the next access. Useful for sweeping bus speeds without rebuilding.

5. Constructor walkthrough

```
bootrom::bootrom(sc_core::sc_module_name name, bootrom_cfg cfg)
: sc_core::sc_module(name)
, size_bytes_p("size_bytes",      cfg.size_bytes,      "...")
, init_file_p("init_file",        cfg.init_file,        "...")
, init_file_format_p("init_file_format", cfg.init_file_format, "...")
, access_delay_ns_p("access_delay_ns", cfg.access_delay_ns, "...")
, reg_socket("reg_socket")
, rst_n_i    ("rst_n_i")
, cfg_(cfg)
{
    // 1. Sync cfg_ with CCI-resolved values
    cfg_.size_bytes      = size_bytes_p.get_value();
    cfg_.init_file       = init_file_p.get_value();
    cfg_.init_file_format = init_file_format_p.get_value();
    cfg_.access_delay_ns = access_delay_ns_p.get_value();

    // 2. Attach provenance metadata (see §4)
    size_bytes_p.add_metadata(...);
    // ...

    // 3. Validate
    if (cfg_.size_bytes == 0)                SC_REPORT_FATAL(...);
    if ((cfg_.size_bytes % bootrom_cfg::WORD_BYTES) != 0) SC_REPORT_FATAL(...);
    if (cfg_.access_delay_ns < 0.0)          SC_REPORT_FATAL(...);

    // 4. Allocate and preload contents
    data_.assign(cfg_.size_bytes, 0);
    if (!cfg_.init_file.empty()) {
        load_preload(data_);                // SC_REPORT_FATAL on error
    }

    // 5. Cache time-typed CCI value
    access_delay_ = sc_core::sc_time(cfg_.access_delay_ns, sc_core::SC_NS);

    // 6. Register TLM callbacks
    reg_socket.register_b_transport (this, &bootrom::b_transport);
    reg_socket.register_transport_dbg(this, &bootrom::transport_dbg);

    // 7. Register the reset-method (structural no-op; see §10)
    SC_METHOD(reset_proc);
    sensitive << rst_n_i;
    dont_initialize();

    // 8. Emit human-readable "instantiated with ..." banner
    SC_REPORT_INFO(name, ...);
}
```

Each step is conceptually small but ordered carefully: the CCI param values must be resolved (step 1) before validation (step 3), the size must be known before allocation (step 4), and the callbacks must be registered (step 6) before elaboration completes.

6. Internal data structures

6.1 CCI parameters (configuration)

Member	Type	Description
<code>size_bytes_p_</code>	<code>cci_param<uint64_t, CCI_IMMUTABLE_PARAM></code>	Total ROM size.
<code>init_file_p_</code>	<code>cci_param<std::string, CCI_IMMUTABLE_PARAM></code>	Preload path.
<code>init_file_format_p_</code>	<code>cci_param<std::string, CCI_IMMUTABLE_PARAM></code>	hex / bin / auto.
<code>access_delay_ns_p_</code>	<code>cci_param<double></code>	TLM annotated delay (mutable).

6.2 State members

Member	Type	Purpose
<code>cfg_</code>	<code>bootrom_cfg</code>	CCI-resolved configuration snapshot, used by the rest of the implementation so we don't re-call <code>*p_.get_value()</code> in hot paths.
<code>data_</code>	<code>std::vector<uint8_t></code>	ROM contents; size = <code>cfg_.size_bytes</code> .
<code>access_delay_</code>	<code>sc_core::sc_time</code>	Cached <code>sc_time</code> form of <code>access_delay_ns_p_</code> ; re-cached on every <code>b_transport</code> .

6.3 Why `std::vector<uint8_t>` rather than `std::vector<uint64_t>`?

The model supports 1-, 2-, 4-, and 8-byte accesses. Storing the backing memory as bytes lets us implement every access width with a single `std::memcpy(buf, data_.data() + off, length)` call, which is both simpler and faster than width-dispatching to 64-bit-word shifts. The 8-byte alignment requirement is enforced separately by the constructor (`size_bytes % WORD_BYTES == 0`).

7. TLM callback implementation

7.1 b_transport

```
void bootrom::b_transport(tlm::tlm_generic_payload& gp,
                          sc_core::sc_time& delay)
{
    // 1. Validate width / alignment / streaming-width
    // 2. Reject byte enables
    // 3. Window check
    // 4. Dispatch on command
    //     - READ : memcpy out of data_
    //     - WRITE: no-op (silent discard per claim C4)
    //     - else : TLM_COMMAND_ERROR_RESPONSE
    // 5. Re-cache access_delay_ from CCI and add to delay
    // 6. set_response_status(TLM_OK_RESPONSE);
}
```

The four pre-checks return `TLM_BURST_ERROR_RESPONSE`, `TLM_BURST_ERROR_RESPONSE`, `TLM_BURST_ERROR_RESPONSE`, and `TLM_BYTE_ENABLE_ERROR_RESPONSE` in that order. The window check returns `TLM_ADDRESS_ERROR_RESPONSE`. The command-switch fallback returns `TLM_COMMAND_ERROR_RESPONSE`. These codes match the canonical error taxonomy used by the PLIC and CLINT models.

7.2 transport_dbg

```
unsigned int bootrom::transport_dbg(tlm::tlm_generic_payload& gp)
{
    // Same validation as b_transport, but on error return 0 (TLM
    // convention) instead of setting a response_status code.
    // Reads: memcpy. Writes: silently ignored.
    // Returns length on success, 0 on validation failure.
}
```

The debug path is a separate function (rather than calling `b_transport` with a zero-time delay) so that debug-time errors do not mutate `response_status` on the supplied `tlm_generic_payload`. This is the same pattern CLINT uses.

7.3 Where the delay comes from

`access_delay_` is re-derived from `access_delay_ns_p_.get_value()` on every transaction:

```
access_delay_ = sc_core::sc_time(access_delay_ns_p_.get_value(),
                                  sc_core::SC_NS);
delay += access_delay_;
```

The CCI broker can mutate `access_delay_ns_p_` from inside any SystemC process or from `sc_main`; the next `b_transport` call sees the new value. The conversion itself (`double` $\rightarrow$ `sc_time`) is cheap and not worth caching across transactions.

8. Preload parser

8.1 Entry point

```
void bootrom::load_preload(std::vector<uint8_t>& data) const
{
    const std::string format = resolve_format(cfg_.init_file_format,
                                              cfg_.init_file);
    if (format == "bin") {
        // open binary, std::ifstream::read up to data.size() bytes
    } else {
        // line-by-line hex parsing (see 8.3)
    }
}
```

`resolve_format` implements the “auto” rule documented in `01_BOOTROM_Specification.md §12.3`: if the user explicitly passed `hex` or `bin`, honour it; otherwise pick `bin` when the filename ends in `.img` / `.bin` (case-insensitive), else `hex`.

8.2 Binary parser

A single `std::ifstream::read` call dumps the on-disk bytes verbatim into `data_`. The destination buffer is already zero-initialised, so a file smaller than `size_bytes` produces a zero-padded tail by construction. A file larger than `size_bytes` is truncated silently (the truncated tail does not affect any reachable address).

8.3 Hex parser

```
std::string line;
unsigned word_idx = 0;
while (std::getline(f, line)) {
    uint64_t v = 0; bool consumed = false;
    if (!parse_hex_line(line, v, consumed)) SC_REPORT_FATAL(...);
    if (!consumed) continue; // blank / comment
    const uint64_t off = word_idx * WORD_BYTES;
    if (off + WORD_BYTES > data.size()) {
        if (v != 0) SC_REPORT_FATAL(...); // sparse-non-zero overflow
        ++word_idx; continue; // tolerate zero overflow
    }
    std::memcpy(data.data() + off, &v, WORD_BYTES);
    ++word_idx;
}
```

`parse_hex_line` accepts `#`-prefixed comments, blank lines, optional `0x` / `0X` prefix, and trailing `# comment` text. Anything else (non-hex characters, multi-token lines) is rejected with `SC_REPORT_FATAL`.

8.4 Sparse-non-zero overflow tolerance

When the hex preload file is longer than `size_bytes`, the loop continues to parse subsequent words but does not write them. A non-zero overflow word triggers `SC_REPORT_FATAL` because it unambiguously indicates the image was sized for a different target. A zero overflow word is tolerated (the SMC tooling routinely emits a 128 KiB hex file for a 64 KiB ROM with the tail zero-padded).

9. Debug back-door API

```
uint64_t dbg_read64(uint64_t off) const;
uint32_t dbg_read32(uint64_t off) const;
unsigned dbg_load_bytes(uint64_t off, const uint8_t* ptr, unsigned len);
void dump_state(std::ostream& os = std::cout) const;
uint64_t size_bytes() const; // CCI-resolved size
```

Method	Role
<code>dbg_read64</code>	Side-effect-free 8-byte peek (returns 0 on OOB / misalignment).
<code>dbg_read32</code>	Side-effect-free 4-byte peek.
<code>dbg_load_bytes</code>	Test-bench-only helper to populate <code>data_</code> without a file.
<code>dump_state</code>	Human-readable snapshot (size, init_file, format, first 32 B).
<code>size_bytes</code>	Returns the CCI-resolved <code>size_bytes</code> value.

`dbg_load_bytes` is the only debug method that mutates state — it is expressly for unit tests that want a deterministic in-memory image without writing a temporary file. Production code should never call it.

10. Reset handling

```
void bootrom::reset_proc()
{
    // Intentionally empty. ROM has no mutable state.
}
```

The method is registered against `rst_n_i` so that the platform top-level can bind reset uniformly across every SMC IP. Leaving the body empty (rather than not registering the method at all) preserves a natural extension point for future revisions — e.g. if a future ROM adds a read-disable lock register, this is where the lock would clear.

11. Error / fatal-report taxonomy

Failure	Mechanism	When
Invalid <code>size_bytes</code> (zero or non-multiple of 8)	<code>SC_REPORT_FATAL</code>	construction
Invalid <code>access_delay_ns</code> (< 0)	<code>SC_REPORT_FATAL</code>	construction
Preload file missing / unreadable	<code>SC_REPORT_FATAL</code>	construction
Malformed hex line	<code>SC_REPORT_FATAL</code>	construction
Sparse-non-zero overflow in hex preload	<code>SC_REPORT_FATAL</code>	construction
Empty preload file	<code>SC_REPORT_FATAL</code>	construction
Bus-level validation failure	<code>TLM_*_ERROR_RESPONSE</code>	run-time
In-range write	<code>TLM_OK_RESPONSE</code> + no-op (claim C4)	run-time

The rule is: **configuration mistakes are fatal at elaboration; bus mistakes return a bus error code**. Never the other way around.

12. Single-driver discipline

The Boot ROM has no output ports, so the single-driver rule that governs PLIC/CLINT output methods does not apply here. The `reset_proc` method is registered solely for IP-suite symmetry; it writes nothing.

13. Performance & memory footprint

Metric	Value (default config)
Heap allocation	<code>size_bytes</code> bytes (default 65 536 B)
Construction time (preload from disk)	dominated by <code>ifstream::read</code> / line parser; < 1 ms for 64 KiB
<code>b_transport</code> cost	one memcpy (≤ 8 B) + one <code>sc_time</code> conversion
<code>transport_dbg</code> cost	one memcpy; no <code>sc_time</code> conversion

For platform-scale benchmarks (millions of fetches per simulated second) the bottleneck is the upstream initiator + quantum keeper, not the ROM model. Enabling DMI (see §16) would reduce per-fetch cost to zero but is not yet wired up.

14. Build & packaging

The module compiles to a static archive `libsmc_bootrom.a` consumable by the top-level SMC SystemC library. See `CMakeLists.txt`:

```
add_library(smc_bootrom STATIC src/bootrom.cpp)
target_include_directories(smc_bootrom PUBLIC include)
target_link_libraries(smc_bootrom PUBLIC SystemC::systemc SystemC::cci)
```

14.1 Dependencies

Dependency	Version	Used for
C++17 baseline; C++20-compliant	gcc 10+ / clang 11+	Standard library + TLM-2.0; builds clean under C++17, C++20, and C++23
Accellera SystemC	2.3.4+ / 3.0.x	<code>sc_core::*</code> , TLM-2.0
Accellera SystemC CCI	1.0.0	<code>cci_configuration</code> header

The CMake glue auto-detects SystemC via `SYSTEMC_HOME` or the `SystemC::systemc` imported target, and CCI via the `CCI_HOME` cache variable / environment variable. See the top-level `CMakeLists.txt` for the exact probe sequence (identical to CLINT).

14.1.1 C++ language standard

The model source is written in portable C++ that compiles cleanly under C++17, C++20, and C++23 — it uses no standard-specific features and is free of constructs removed or deprecated by C++20 (no `std::result_of`, throwing `std::allocator`, `std::is_pod`, comma-in-subscript, etc.).

The build does **not** hard-code a standard. Instead `SmcSystemCStd.cmake` (`smc_detect_systemc_cxx_std()`) probes the linked SystemC library's `sc_api_version_*_cxxNNNNNNL` guard symbol and sets `CMAKE_CXX_STANDARD` to match — because Accellera SystemC's ABI changes per language standard, the consumer must compile with the *same* standard the library was built with, or the link fails with an undefined `sc_api_version_*_cxx20XXXX` symbol.

With the project's reference toolchain (SystemC 3.0.2 + CCI built with C++20), the Boot ROM is therefore **built and verified under C++20**:

```
# verified: clean build under C++20 (-Wall -Wextra -Wpedantic, 0 warnings)
cmake -S . -B build -DSMC_CXX_STANDARD=20 # or rely on auto-detect
cmake --build build -j
# bootrom_tb / bootrom_bin_tb / bootrom_neg_tb → ALL TESTS PASSED
```

Pass `-DSMC_CXX_STANDARD=<17|20|23>` to force a specific standard (e.g. to match a Homebrew SystemC built with C++17).

14.2 Optional instrumentation

```
./run_tests.sh --asan # AddressSanitizer (incl. LSan on Linux)
./run_tests.sh --coverage # LLVM source-based coverage (Clang) or gcov (GCC)
```

The two are mutually exclusive (incompatible runtime instrumentation). Each uses an isolated build directory (`build_asan/` / `build_cov/`) so they never invalidate the plain Release build's CMake cache.

15. Coding-style notes

- Written to a C++17 baseline but **C++20-compliant** (and C++23-clean); the build standard is auto-matched to the linked SystemC by `SmcSystemCStd.cmake` and is C++20 with the reference toolchain (see §14.1.1). No exceptions in production paths except where the standard library throws (e.g. `std::stoull` on parse error — caught and converted to a controlled `SC_REPORT_FATAL`).
- `-Wall -Wextra -Wpedantic -Wno-deprecated-declarations`. The deprecation suppression is to silence Accellera SystemC's own internal deprecations; the project source itself does not invoke any deprecated API.
- All public symbols live in namespace `smc` so the eventual top-level `libsmc.a` does not collide with other IPs.
- Internal helpers (`is_supported_width`, `is_aligned`, `to_lower`, `ends_with_ci`) live in anonymous namespaces inside `bootrom.cpp`.
- Comments document **why**, not **what** — the code is its own description of what; comments are reserved for non-obvious intent (e.g. the sparse-non-zero-overflow tolerance in §8.4).

16. Future work

- **DMI fast-path.** A read-only ROM is the canonical DMI candidate; granting `dmi_allowed = true` on the first read would skip the per-fetch `b_transport` and bring the cost to zero. Held back only for parity with PLIC/CLINT.
 - **Approximately-Timed (AT) protocol.** Not currently planned; the RTL only emits 1-cycle `rvalid` so the LT abstraction is a perfect match.
 - **CRC / hash sanity check.** When loading a binary image, optionally verify a manifest hash so silent image-corruption is caught at elaboration. Would require a new optional CCI string parameter (`init_file_sha256`).
 - **Secure-region / key-disable lock.** Tracked against the next SEP RDL revision; would add a CCI flag and per-region access policy.
 - **Coverage of `SC_REPORT_FATAL` paths.** Achieving 100% line coverage requires forking a child process per fatal scenario; the current test plan documents this trade-off explicitly (see `03_BOOTROM_Test_Plan.md §9.2`).
-

End of document.